



Arquiteturas Avançadas de Computadores

2025

André Pereira

Background to this course
(slides from several sources)



- **Documentation & oral lectures/classes**

- Documentation (slides, lab guides, books, reports...): English
- Oral presentations: optional, but usually Portuguese

- **Expected background**

- Imperative Programming and Computer Architecture, Parallel Programming / Computing
- Lab skills in Unix, C/C++ programming and debugging

- **Assessment**

- 1x paper analysis an HPC subject 15 %
 - Followed by a presentation and group discussion
- 1x practical work assignment 50 %
- 1x Written test 35 %

Focus on this course: performance engineering



How:

- understanding the organisation of a computer system (its architecture)
- plan and code efficient algorithms & data structures
- profiling & evaluating (through measurements) the execution efficiency

Where in the hardware:

- in sequential code and memory hierarchy
 - Revisiting some background concepts
- w/ support for thread parallelism
 - new threading technologies (C++ threads – let 's avoid pthreads)
 - multithreading in multiple devices
- w/ computing accelerators (GPU, ...)
 - using them simultaneously with CPUs
- w/ support for task parallelism
 - multiprocessing in a computer cluster

Parallel Computing – What's Different?



- Similar foundations
 - Parallel Computing touched on several subjects with little detail
 - ACA provides deeper understanding on some of them
- More focus on
 - ...

Parallel Computing – What's Different?



- Similar foundations
- More focus on
 - Profiling with tools used in supercomputing
 - Although they may use same concepts and underlying interfaces
 - Advanced features
 - Async GPU usage, DMA, parallelisation frameworks
 - Simultaneous CPU - GPU computing
 - Load Scheduling

Tentative Course Plan



Semear	Data	Teórica	Prática Laboratorial
1	15/09 a 20/09		
2	22/09 a 27/09	Apresentação. Intro ao HPC. Overview top-down de um servidor heterogêneo e organização básica de um supercomputador.	2h
3	29/09 a 04/10	Introdução aos performance counters. Profiling (PAPI/TAU, VTune, e Advisor).	1h30
4	06/10 a 11/10	Multicore vs Manycore	1h30
5	13/10 a 18/10	Roofline: modelação de performance	1h30
6	20/10 a 25/10	Arquitetura ARM: o A64FX	1h
7	27/10 a 01/11	Defesa do 1º trabalho prático	3h
8	03/11 a 08/11	GPUs e processadores heterogêneos	1h
9	10/11 a 15/11	Metodologias de teste de código	1h
10	17/11 a 22/11	Cargas regulares vs irregulares e escalonamento	1h
11	24/11 a 29/11	Apoio aos projetos do 2º trabalho prático	3h
12	01/12 a 06/12	Computação heterogênea	1h
13	08/12 a 13/12	PCAM? Energia na supercomputação?	1h
14	15/12 a 20/12	Panorama da supercomputação europeia	2h
		Teste	3h
		Entrega do 2º trabalho dia 4 de Janeiro	Defesa do 2º trabalho dia 9 de Janeiro

Key textbook (1)

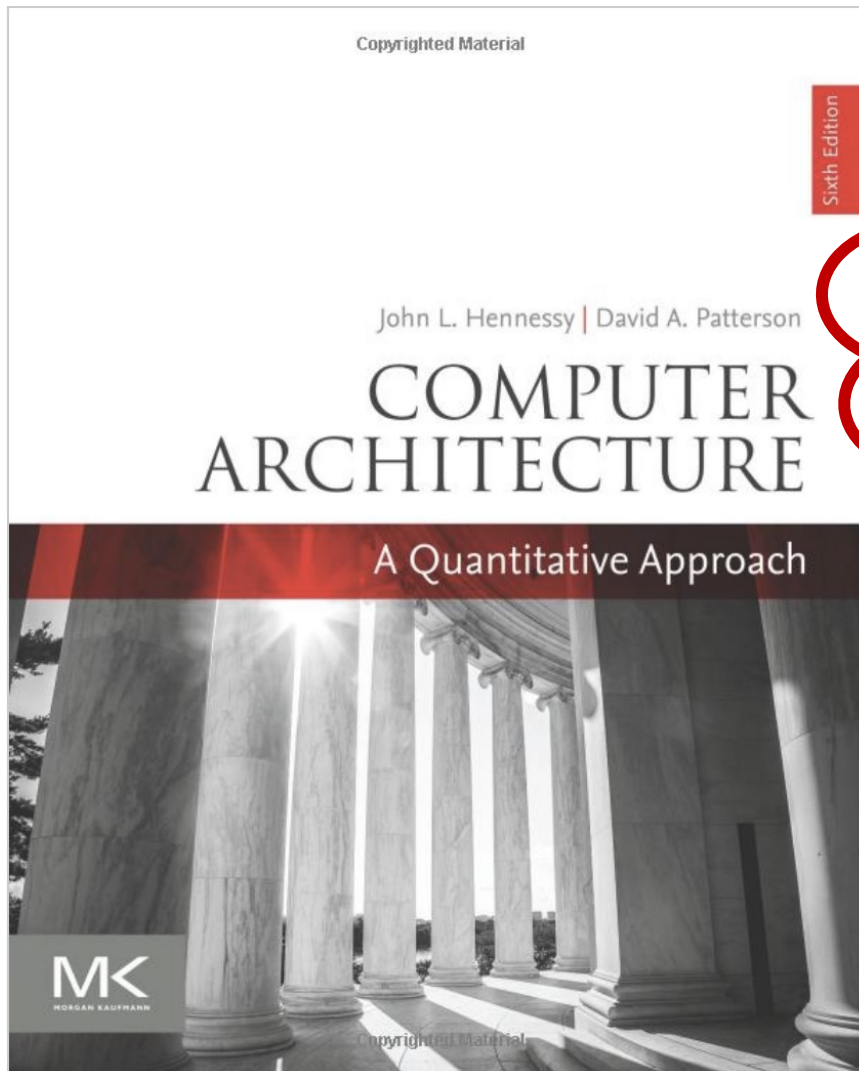


Table of Contents

Printed Text

1. Fundamentals of Quantitative Design and Analysis
2. Memory Hierarchy Design
3. Instruction-Level Parallelism and Its Exploitation
4. Data-Level Parallelism in Vector, SIMD, and GPU Architectures
5. Multiprocessors and Thread-Level Parallelism
6. The Warehouse-Scale Computer
7. Domain Specific Architectures
 - A. Instruction Set Principles
 - B. Review of Memory Hierarchy
 - C. Pipelining, Basic and Intermediate Concepts

Online

- D. Storage Systems
- E. Embedded Systems
- F. Interconnection Networks
- G. Vector Processors
- H. Hardware and Software for VLIW and EPIC
- I. Large-Scale Multiprocessors and Scientific Applications
- J. Computer Arithmetic
- K. Survey of Instruction Set Architectures
- L. Advanced Concepts on Address Translation
- M. Historical Perspectives and References

e-book...

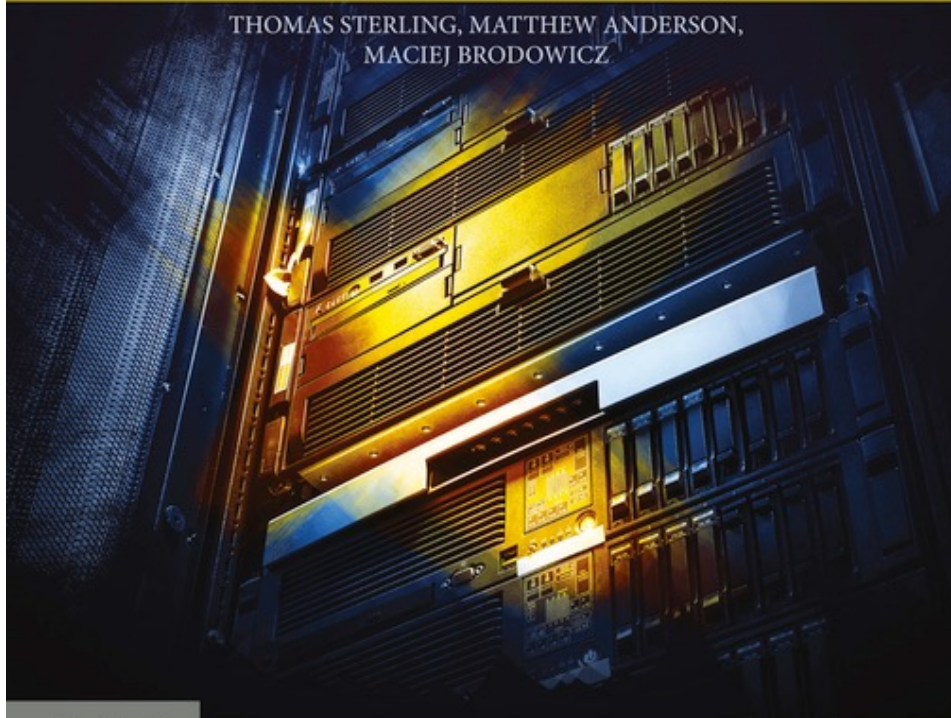
Key textbook (2)



HIGH PERFORMANCE COMPUTING

MODERN SYSTEMS AND PRACTICES

THOMAS STERLING, MATTHEW ANDERSON,
MACIEJ BRODOWICZ



MK
MORGAN KAUFMANN

FOREWORD BY C. GORDON BELL

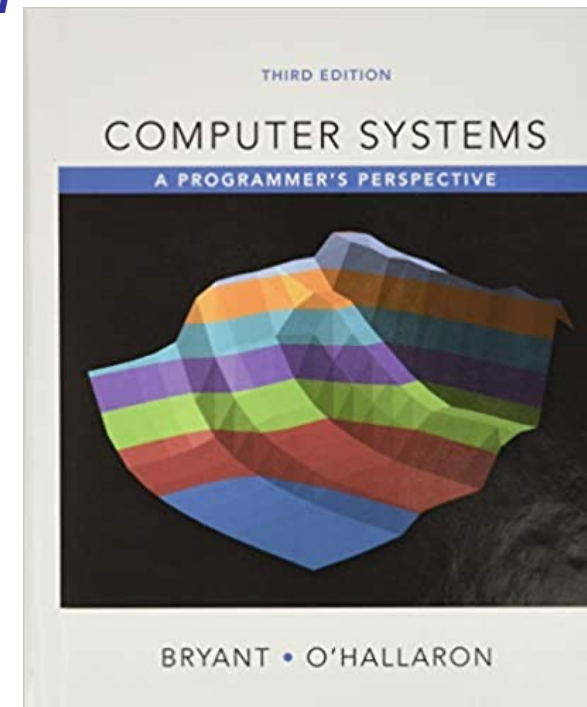
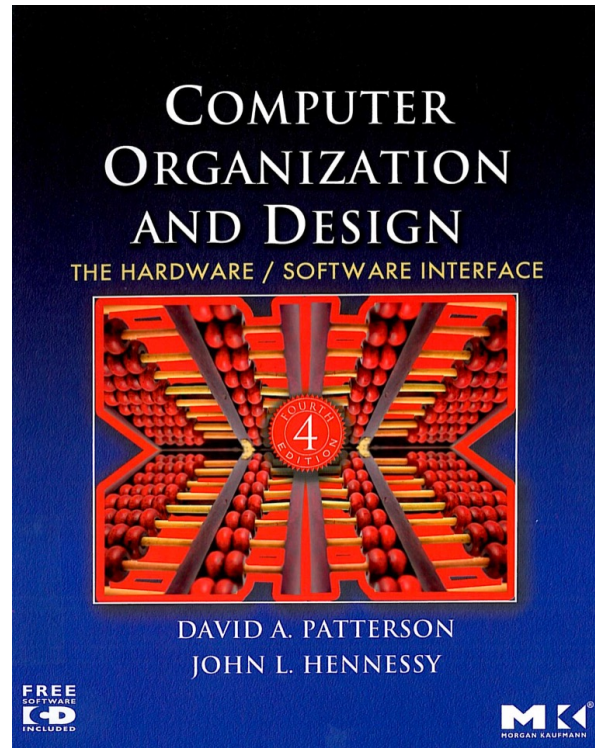
Background concepts from a basic Computer Systems course



Some notes/comments for this course:

– some slides are borrowed from

and some from



more details at
<http://gec.di.uminho.pt/miei/sc2021/>

Measuring Performance

- Typical performance metrics:
 - Response time
 - Throughput
- Speedup of X relative to Y
 - $\text{Execution time}_Y / \text{Execution time}_X$
- Execution time
 - Wall clock time: includes all system overheads
 - CPU time: only computation time
- Benchmarks
 - Kernels (e.g. matrix multiply)
 - Toy programs (e.g. sorting)
 - Synthetic benchmarks (e.g. Dhrystone)
 - Benchmark suites (e.g. SPEC06fp, TPC-C)

Revisiting....

Lei de Amdahl

O ganho no desempenho – *speedup* –

obtido com a melhoria do tempo de execução de uma parte do sistema, está limitado pela fração de tempo que essa parte do sistema pode ser usada.

$$\text{Speedup}_{\text{overall}} = \frac{\text{Tempo_exec}_{\text{antigo}}}{\text{Tempo_exec}_{\text{novo}}} = \frac{1}{\sum (f_i / s_i)}$$

f_i - frações com melhoria s_i
 s_i - *speedup* de cada fração

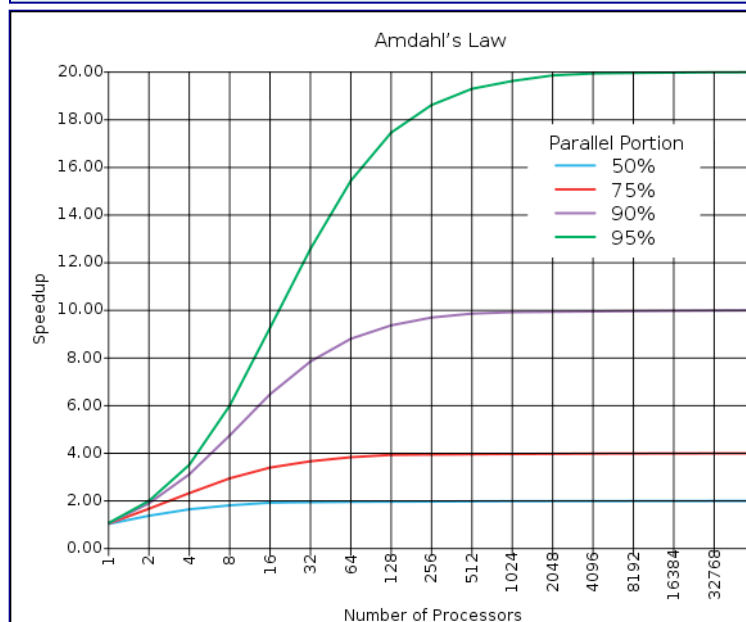
Ex.1: Se 10% de um programa executa 90x mais rápido

Overall speedup = 1.11

Ex.2: Se 90% de um prog executa 90x mais rápido

Overall speedup = 9.09

Paralelismo:
se $N_{\text{proc}} \equiv \text{speedup}$, trocar s_i por N_{proc}



Amdahl's Law



$$\text{Overall Speedup} = \frac{\text{Old execution time}}{\text{New execution time}}$$

$$= \frac{1}{\left((1 - \text{Fraction}_{\text{enhanced}}) + \frac{\text{Fraction}_{\text{enhanced}}}{\text{Speedup}_{\text{enhanced}}} \right)}$$

$$\text{Speedup} = \frac{1}{(1 - p) + p/N}$$

$$\text{Speedup}(N) = \frac{1}{(1-P) + \frac{P}{N}}$$

Serial part of job =
1 (100%) - Parallel part

Parallel part is divided
up by N workers

Principles of Computer Design

- The Processor Performance Equation

$$\text{CPU time} = \text{CPU clock cycles for a program} \times \text{Clock cycle time}$$

$$\text{CPU time} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

$$\text{CPI} = \frac{\text{CPU clock cycles for a program}}{\text{Instruction count}}$$

$$\text{CPU time} = \text{Instruction count} \times \text{Cycles per instruction} \times \text{Clock cycle time}$$

$$\frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Clock cycles}}{\text{Instruction}} \times \frac{\text{Seconds}}{\text{Clock cycle}} = \frac{\text{Seconds}}{\text{Program}} = \text{CPU time}$$

Background for Advanced Architectures in Parallel Computing



Key concepts to revise:

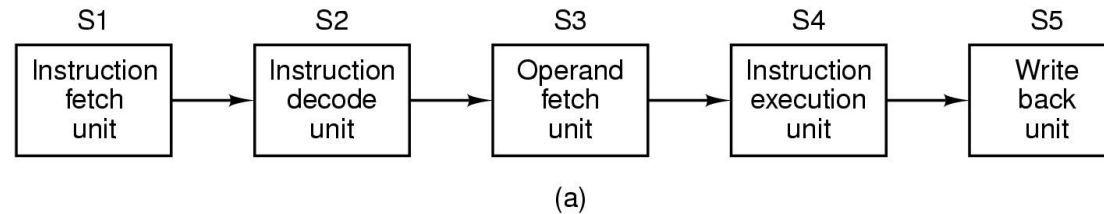
- *numerical data representation (integers & FP)*
- *ISA (Instruction Set Architecture)*
- *how C compilers generate code (a look into assembly code)*
 - *how scalar and structured data are allocated*
 - *how control structures are implemented*
 - *how to call/return from function/procedures*
 - *what architecture features impact performance*
- *improvements to enhance performance in a single PU*
 - *ILP: pipeline, multiple issue, ...*
 - *data parallelism: SIMD/vector processing, ...*
 - *thread-level parallelism*
 - *memory hierarchy: cache levels, ...*

Keyword: parallelism

Paralelismo no processador

Exemplo 1

Exemplo de pipeline

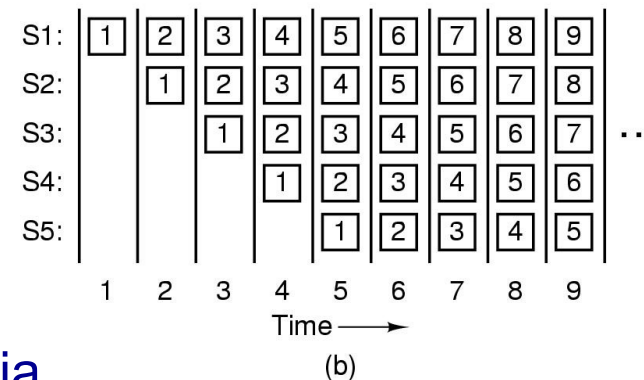


Objetivo

- CPI = 1

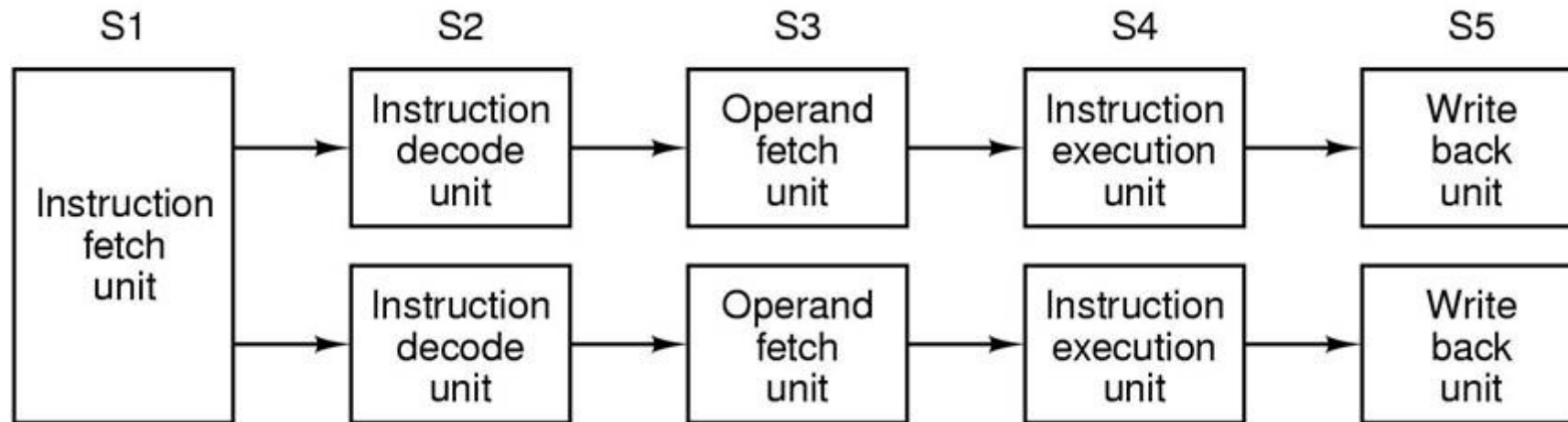
Problemas:

- dependências de dados
- latências nos acessos à memória
- saltos condicionais; propostas de solução para minimizar perdas:
 - executar sempre a instrução "que se segue"
 - usar o historial dos saltos anteriores (1 ou mais bits)
 - executar os 2 percursos alternativos até à tomada de decisão



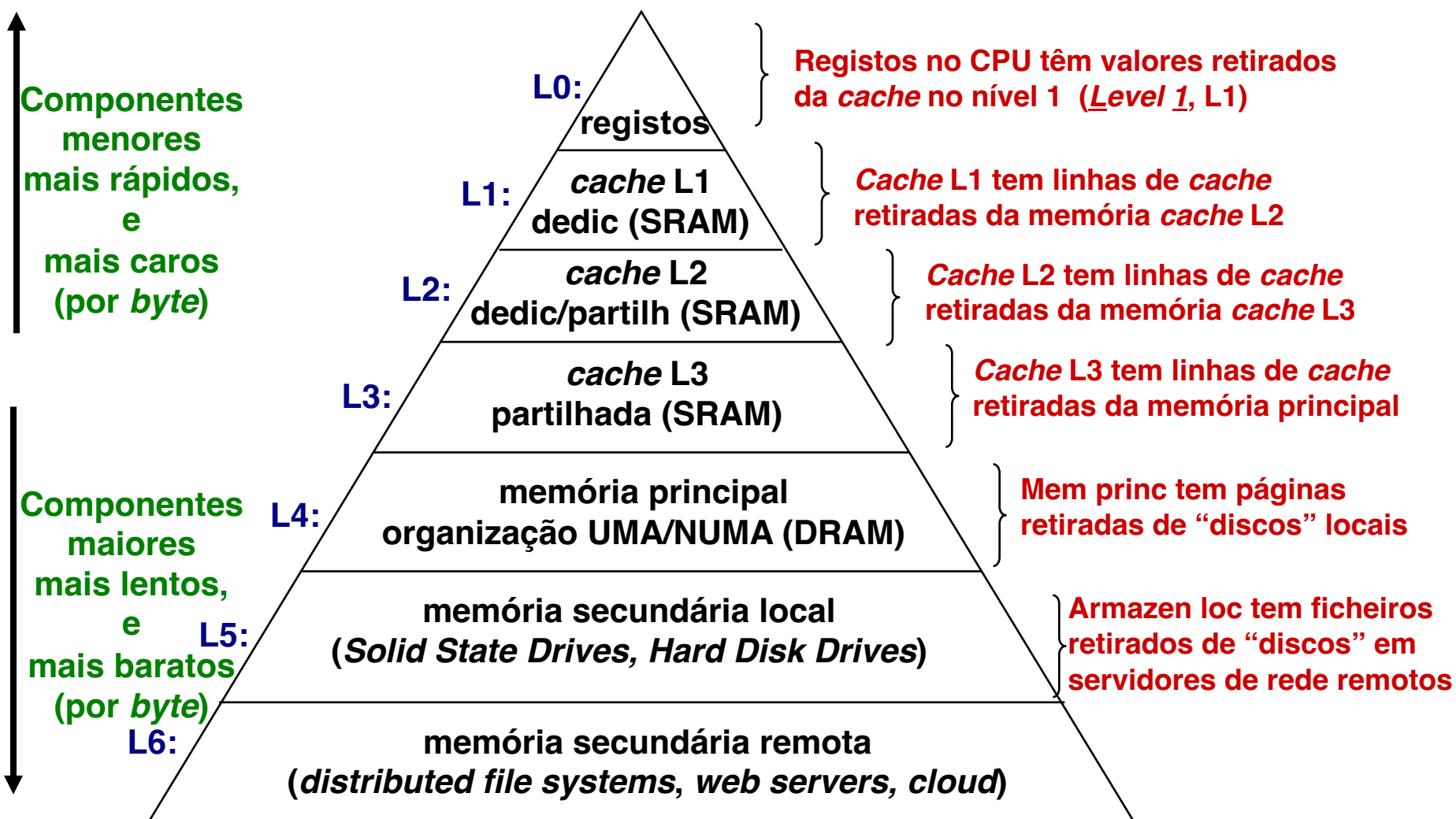


Exemplo de superescalaridade (2 vias)



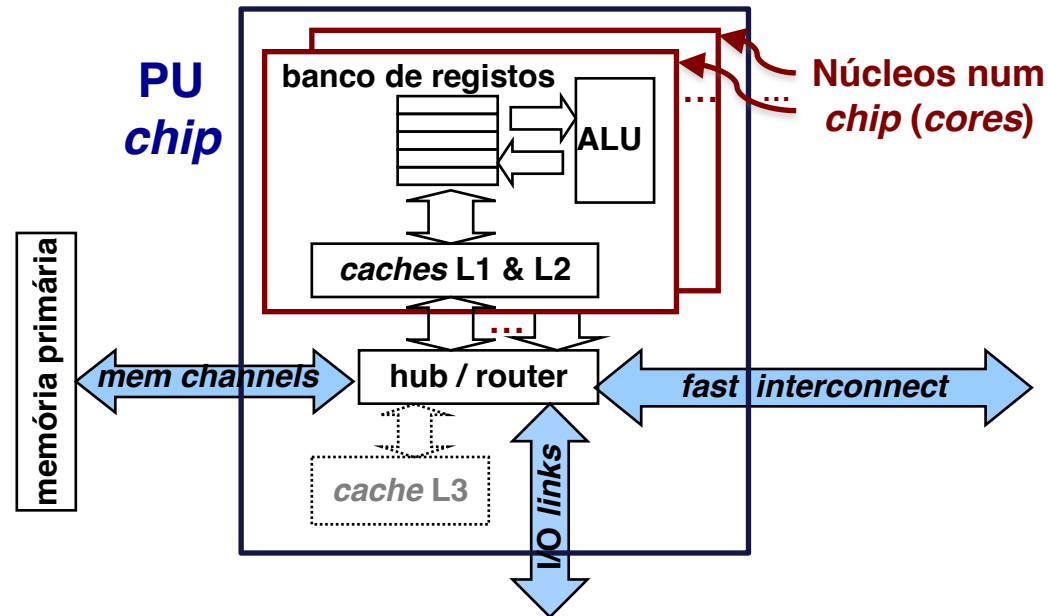
Revisiting....

Organização hierárquica da memória



Revisiting....

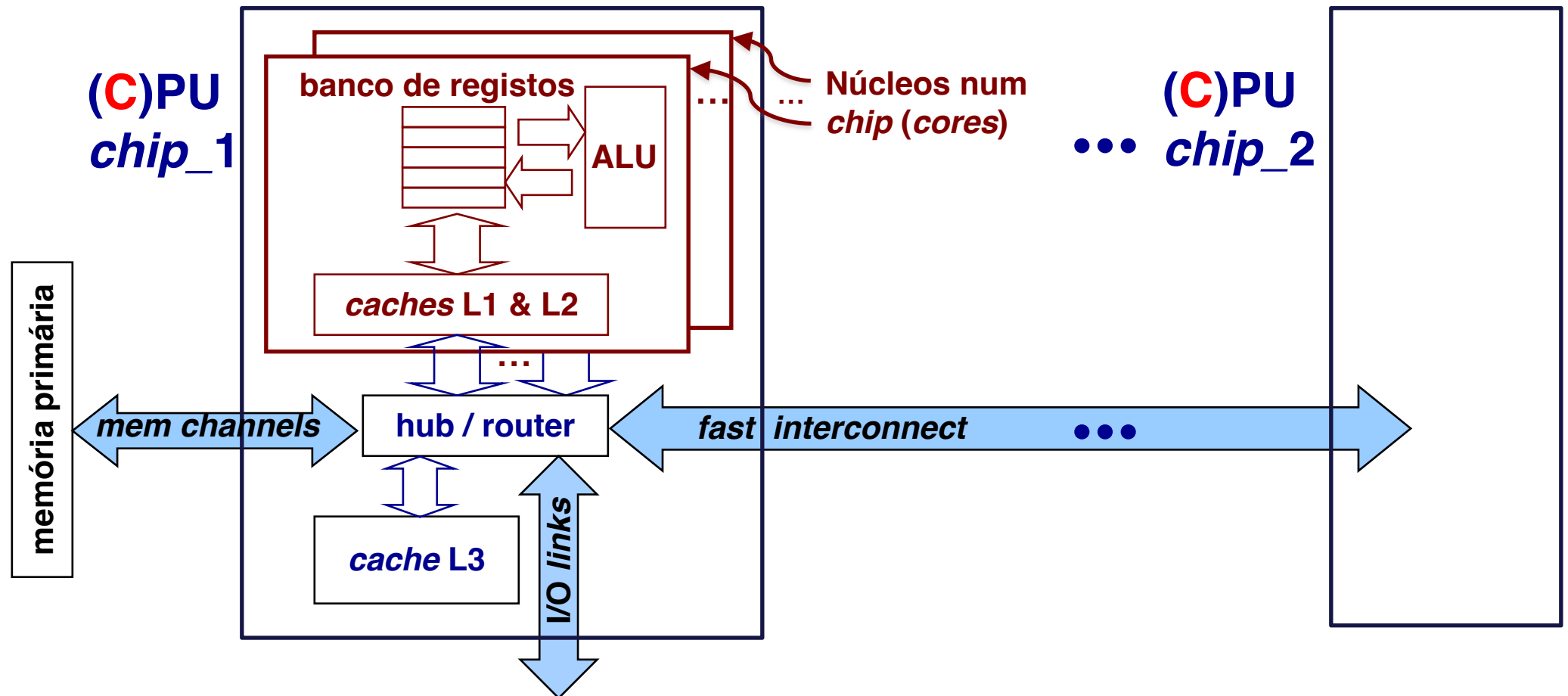
A cache em arquiteturas multicore



Notas:

- as *caches* L1 de dados e de instruções são normalmente distintas
- as *caches* L2 em *multicores* podem ser partilhadas por outras *cores*
- muitos *cores* partilhando uma única memória traz complexidades:
 - manutenção da coerência da informação nas *caches*
 - encaminhamento e partilha dos circuitos de acesso à memória

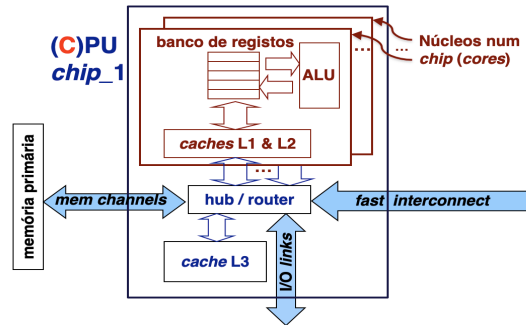
Multicore architecture in a server system



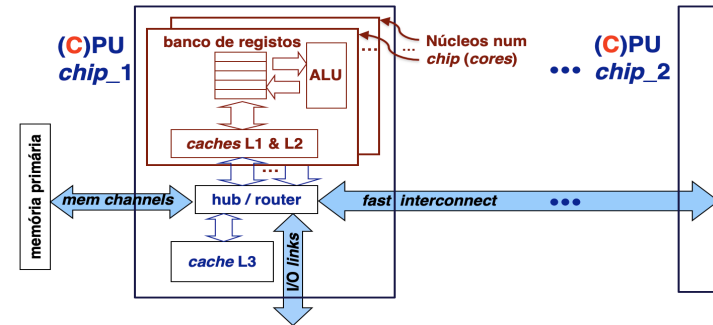
Memory systems in current computer architectures



Shared memory systems

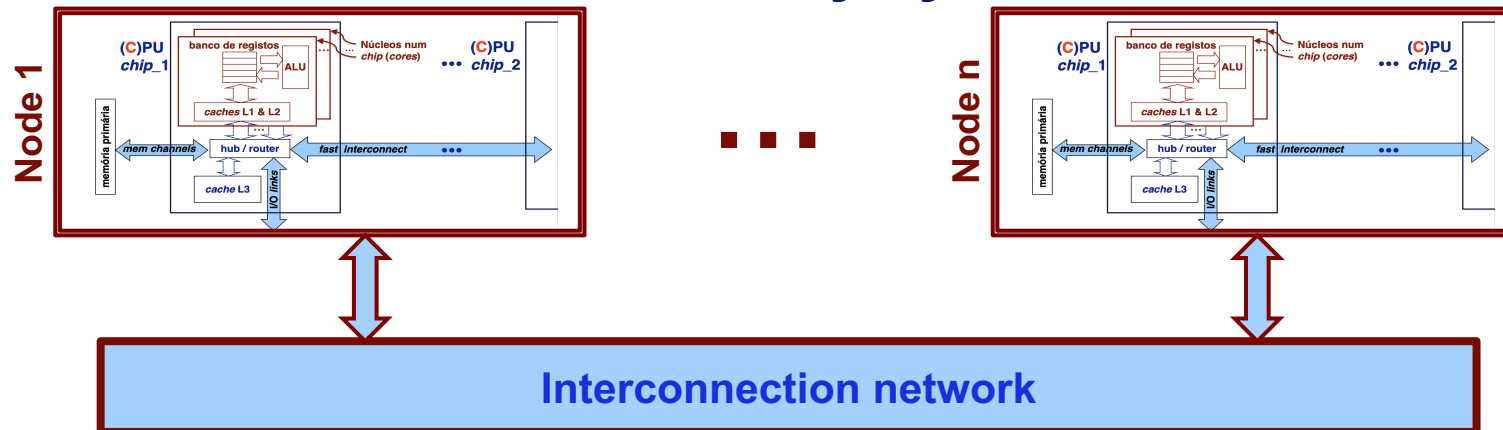


UMA: Uniform Memory Access



NUMA: Non-Uniform Memory Access

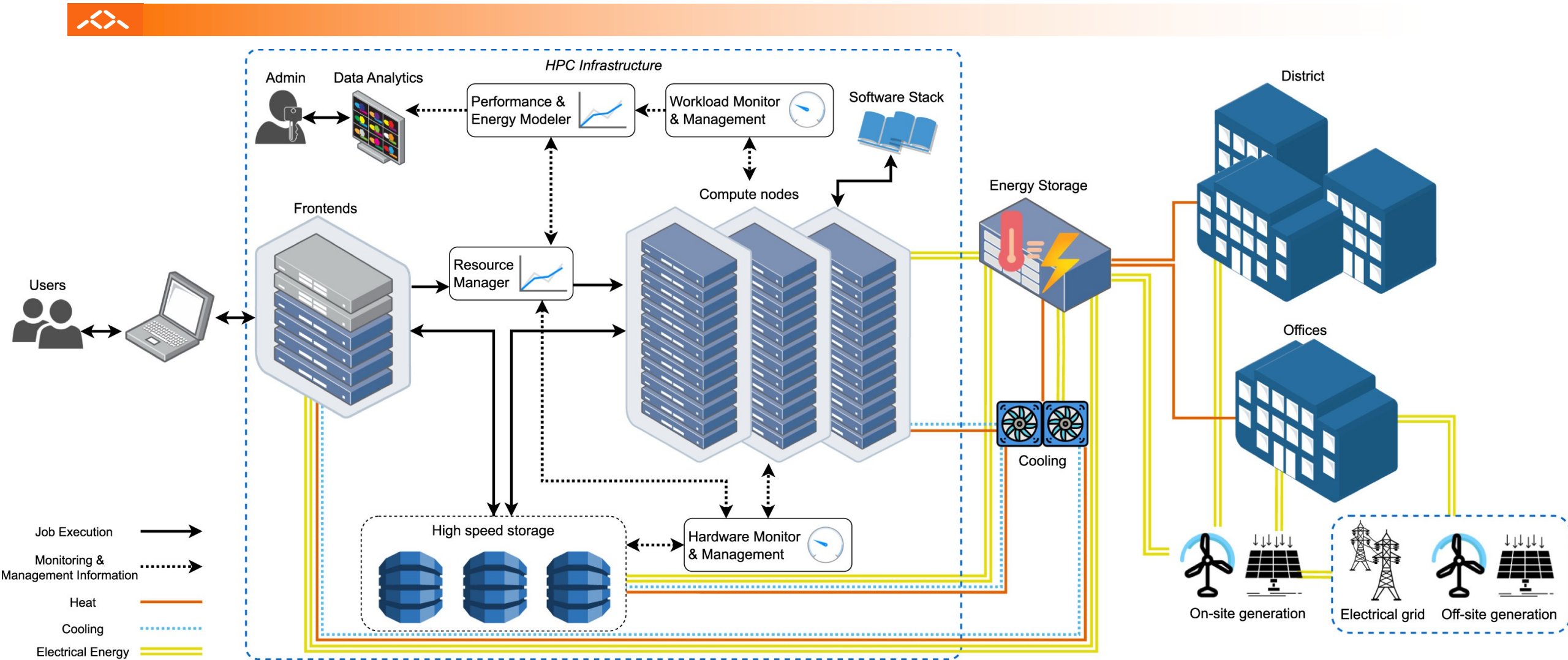
Distributed shared-memory systems



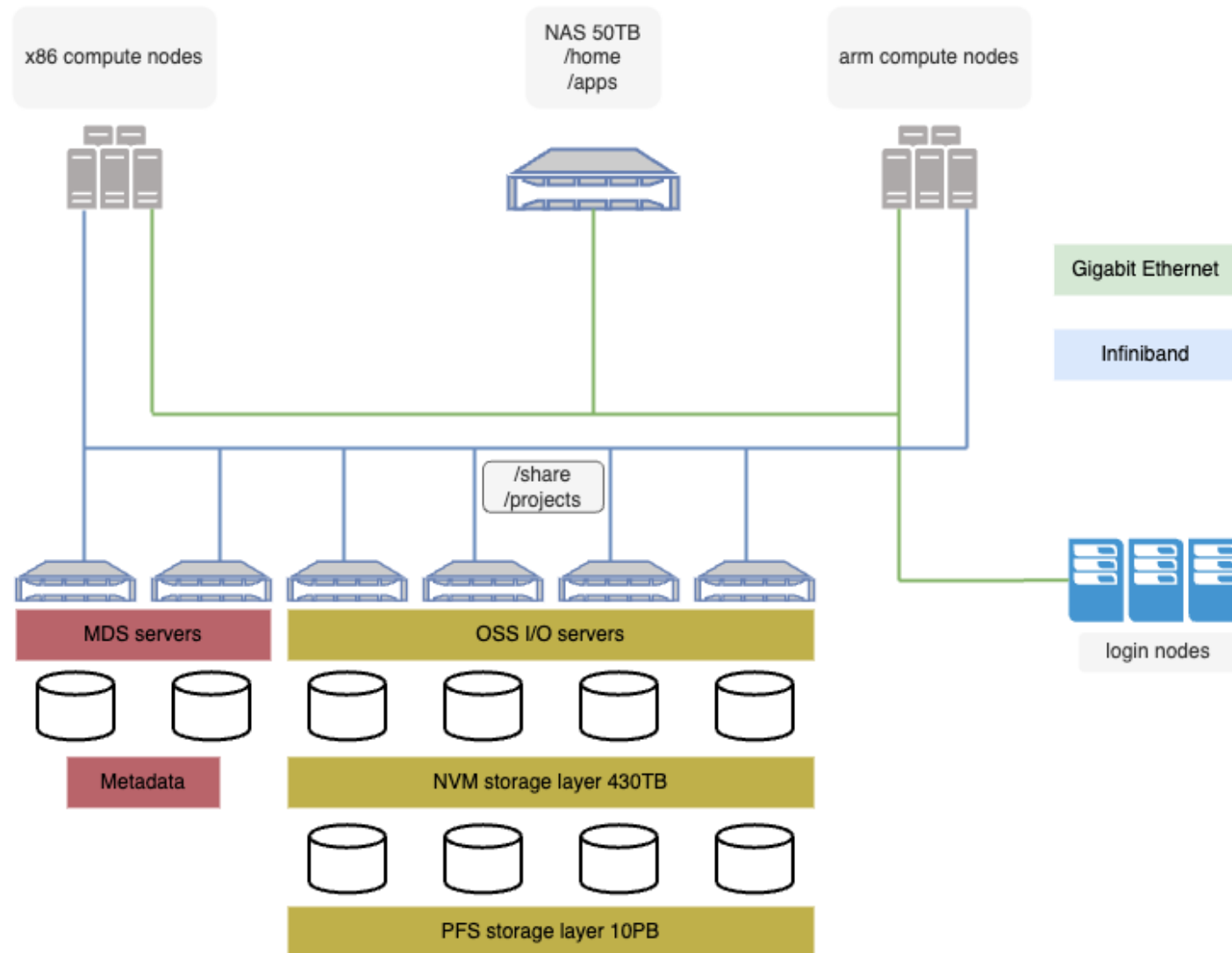


HPC – What for?

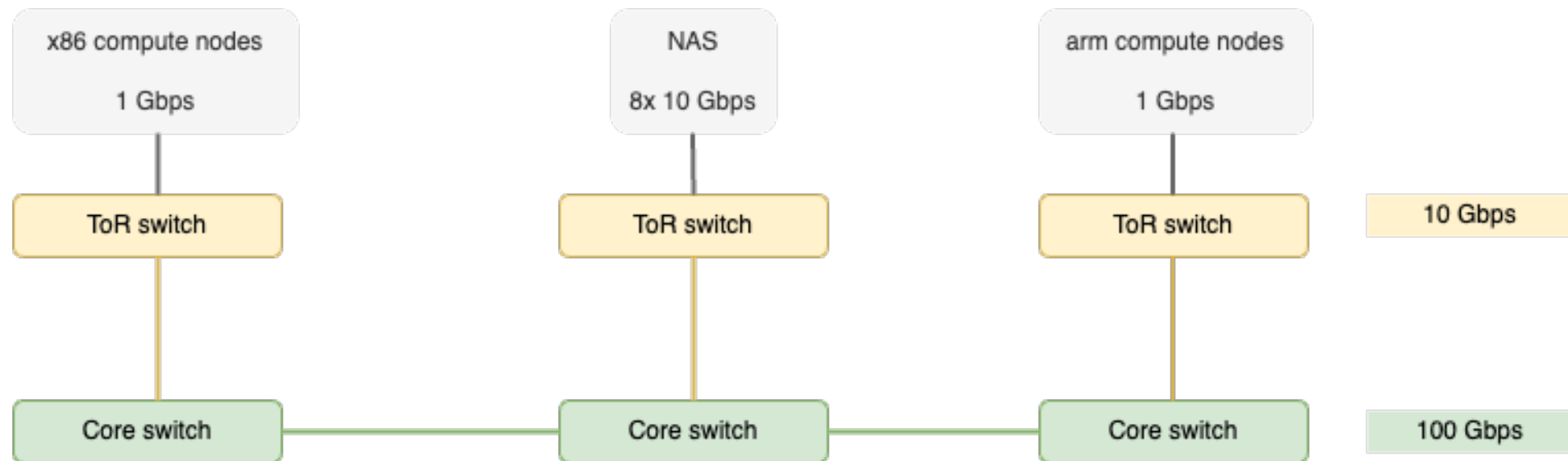
Supercomputing Architecture – an Holistic View



Supercomputing Architecture – Deucalion

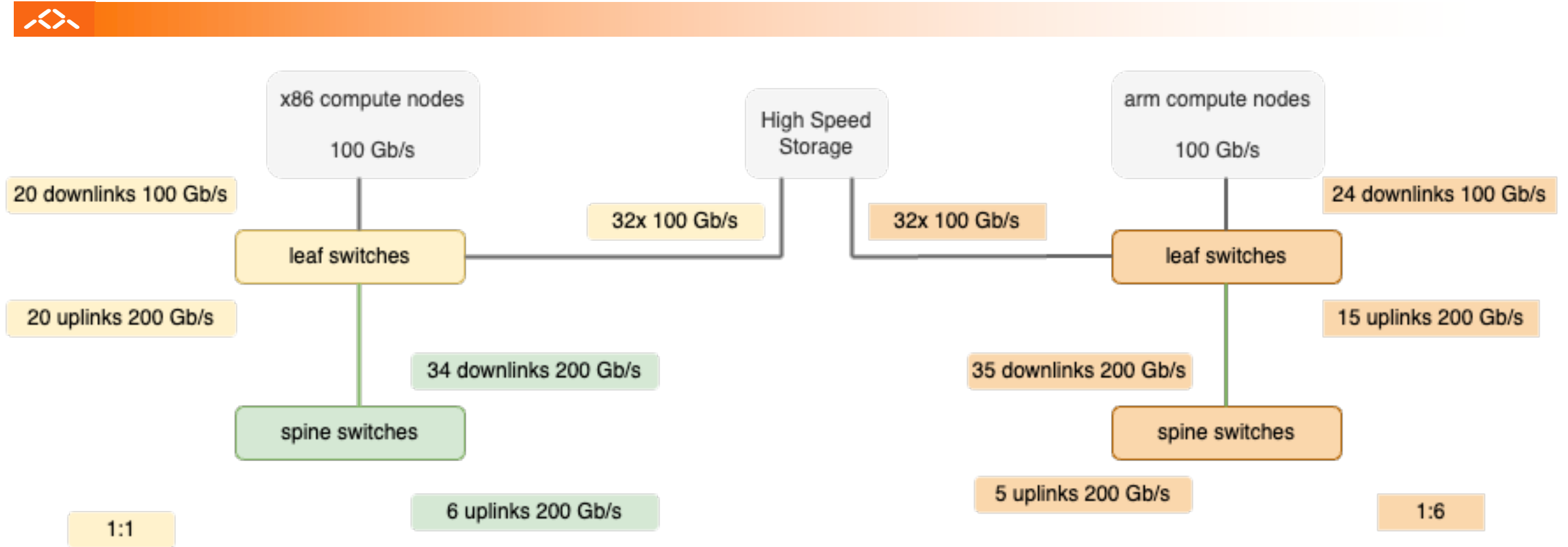


Supercomputing Architecture – Deucalion



Ethernet Network

Supercomputing Architecture – Deucalion



Ethernet Network